

AI Discussions on Bluesky: Topics, Engagement and Follow Networks

COMP3020 Social Web Analytics - Group Project

Group 6

Kimmy Le (22193952) – contribution 30%

Phoebe Le (22122352) – contribution 40%

Quinn Nguyen (22148559) – contribution 30%

9 October 2026

Contents

1	Introduction	2
2	Data Collection	2
2.1	Method	2
2.2	Data volume and variables	3
2.3	Limitations of the collection	3
3	Text Analysis and Visualisation	3
3.1	Cleaning	3
3.2	Corpus, pre-processing and document-term matrix	4
3.3	Frequent words	5
3.4	Post length	5
3.5	TF-IDF representation	7
4	RQ1: Topics in AI Posts (Clustering)	7
4.1	Representation, distance and number of clusters	7
4.2	Cluster themes	9
4.3	Interpretation	11
4.4	Do the clusters simply repeat the search terms?	11
5	RQ2: Does Engagement Differ Between Topics?	12
5.1	Hypotheses and choice of test	12
5.2	Comparing all seven topics	14
5.3	Interpretation	15

6	RQ3: Follow Network of AI Post Authors	15
6.1	Network definition	15
6.2	Network properties	15
6.3	Visualisation	16
6.4	Centrality	17
6.5	Interpretation	20
6.6	Linking the network to engagement (RQ2 and RQ3)	20
6.7	Effect of data collection on the network	22
7	Overall Findings	22
8	Limitations	22
9	References	23

1 Introduction

Artificial intelligence (AI) is one of the most discussed topics on social media. This project studies a sample of public **Bluesky** posts about AI and asks three connected research questions:

1. **RQ1 (Text analysis and clustering):** What are the main topics in AI-related posts on Bluesky?
2. **RQ2 (Hypothesis testing):** Does user engagement differ between these topics?
3. **RQ3 (Network analysis):** How are the authors of AI posts connected through follow relationships, and which accounts are central?

The questions build on each other: the topics found in RQ1 define the groups compared in RQ2, and the same topics are used to colour the authors in the RQ3 network.

Code and data repository: <https://github.com/howaboutlinh/COMP3020-Bluesky-AI-Analysis>

The repository contains the five R scripts (**scripts/01–05**), the collected data (**data/raw/**) and the processed data (**data/processed/**). This report re-runs the analysis from the saved data; it does not make new API requests.

2 Data Collection

2.1 Method

Data were collected in R with the **atrrr** package, which was used in the COMP3020 labs (Modules 4, 5 and 7).

- **Posts:** `search_post()` was called with four search terms: *artificial intelligence*, *machine learning*, *ChatGPT* and *generative AI*, sorted by latest, with `limit = 250` per term. The API returns results in pages of 100, so about 300 posts were returned per term.
- **Follow relationships:** for every author of a collected post, `get_follows()` retrieved the accounts that author follows (requested limit 200, returned in pages of up to about 300), following Module 7 Part 6.

```

# Script 01 (not re-run here): search posts and collect follows.
library(atrrr)
posts_ai = search_post("artificial intelligence", sort = "latest", limit = 250)
# ... same for "machine learning", "ChatGPT", "generative AI" ...
posts = rbind(posts_ai, posts_ml, posts_chatgpt, posts_genai)
posts = posts[!duplicated(posts$uri), ] # one copy of each post

authors = unique(posts$author_handle)
more_friends = lapply(authors, get_follows, limit = 200)
more_friends_handles = lapply(more_friends, `[`, "actor_handle")
el_friends = lapply(seq_along(authors), function(i) {
  cbind(from = rep(authors[i], length(more_friends_handles[[i]])),
        to = more_friends_handles[[i]])
})
el = unique(do.call(rbind, el_friends))

```

2.2 Data volume and variables

Item	Value
Posts returned (AI / ML / ChatGPT / GenAI)	299 / 300 / 299 / 291 = 1,189
Unique posts after removing duplicate URIs	1,168
Distinct authors	887
Time period	7 Oct 2026 15:24 UTC to 8 Oct 2026 16:11 UTC (about 25 hours); 124 posts had no timestamp
Follow relationships collected	118,988 directed edges

Variables kept for each post: `uri` (post id), `author_handle`, `text`, `created_at`, `like_count`, `repost_count`, `reply_count`, `quote_count` and `search_keyword`. Each follow relationship is a pair (`from`, `to`) meaning *from follows to*.

2.3 Limitations of the collection

- Keyword search only finds posts that use these exact terms, and some posts use them only in passing.
- The sample covers about one day, so it reflects the news of that day (for example, an AP-NORC poll released on 8 October).
- The search API returns the latest posts, not a random sample.
- At most about 300 follows were collected per author, so some follow relationships are missing.

3 Text Analysis and Visualisation

3.1 Cleaning

Posts with missing or empty text were removed. Many posts were copies of the same message (bots or reposted headlines), so posts with the same text, ignoring links and letter case, were also removed.

```

library(tm)
library(SnowballC)
library(wordcloud)

posts = read.csv("data/raw/bluesky_posts.csv")

```

```

n.collected = nrow(posts)

# Remove posts with missing or empty text.
posts = posts[!is.na(posts$text), ]
posts = posts[trimws(posts$text) != "", ]
n.nonempty = nrow(posts)

# Web addresses: "http(s)://...", "www..." and shortened links
# such as "nytimes.com/2026/..." (a word with a dot followed by "/").
url.pattern = paste0("https?:/[^\[:space:]]+|www\\.[^\[:space:]]+|",
                     "[^\[:space:]]+\\.[^\[:space:]]+/[^\[:space:]]*")

# Remove repeated text, ignoring links and letter case.
compare.text = tolower(trimws(gsub(url.pattern, "", posts$text)))
posts = posts[!duplicated(compare.text), ]
n.unique = nrow(posts)
c(collected = n.collected, non_empty = n.nonempty, unique_text = n.unique)

```

```

#>   collected   non_empty unique_text
#>      1168      1138      1051

```

3.2 Corpus, pre-processing and document-term matrix

The text was converted into a corpus and cleaned with the same steps as Modules 4 and 6. The four search terms appear in almost every post and would dominate the results, so they were removed together with the English stop words (Module 6 removes the search names for the same reason).

```

corpus = Corpus(VectorSource(posts$text))
remove_urls = content_transformer(function(x) gsub(url.pattern, " ", x))
corpus = tm_map(corpus, remove_urls)
corpus = tm_map(corpus, function(x) iconv(x, to = "ASCII", sub = " "))
corpus = tm_map(corpus, removeNumbers)
corpus = tm_map(corpus, removePunctuation)
corpus = tm_map(corpus, stripWhitespace)
corpus = tm_map(corpus, tolower)
search_words = c("artificial", "intelligence", "machine", "learning",
                  "chatgpt", "generative")
corpus = tm_map(corpus, removeWords, c(stopwords("english"), search_words))
corpus = tm_map(corpus, stemDocument)

# Document-term matrix: rows = posts, columns = terms.
Bluesky.matrix = as.matrix(DocumentTermMatrix(corpus))

# Remove posts with no terms left after cleaning.
keep = rowSums(Bluesky.matrix) > 0
Bluesky.matrix = Bluesky.matrix[keep, ]
posts = posts[keep, ]
dim(Bluesky.matrix)

```

```

#> [1] 1031 5483

```

After cleaning, **1031 posts** and **5483 terms** remain. `DocumentTermMatrix()` ignores words shorter than three letters, so “AI” itself is not a term. This is acceptable because every post was retrieved with an AI search term.

3.3 Frequent words

```
w = colSums(Bluesky.matrix)
o = order(w, decreasing = TRUE)[1:20]
par(mar = c(4, 7, 2, 1))
barplot(rev(w[o]), names.arg = rev(colnames(Bluesky.matrix)[o]),
        horiz = TRUE, las = 1, col = "steelblue",
        main = "Top 20 Most Frequent Terms", xlab = "Frequency")
```

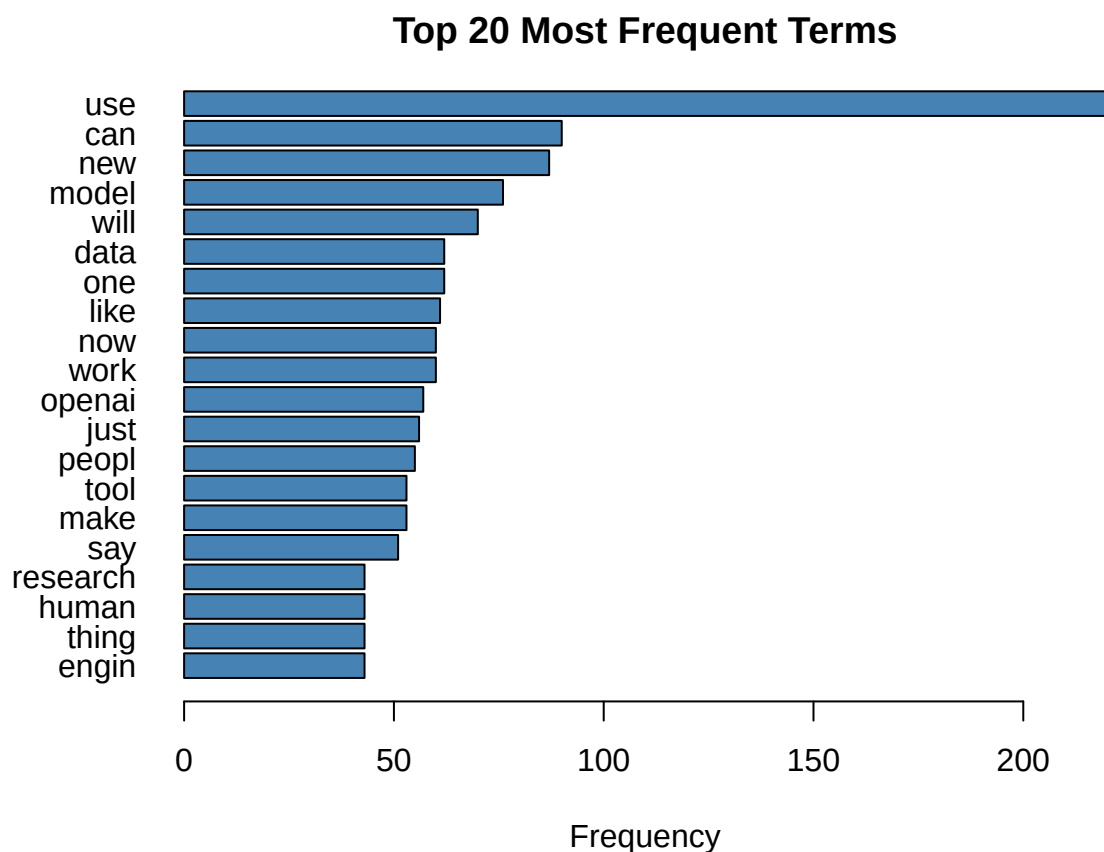


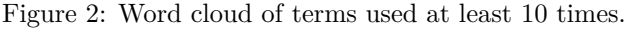
Figure 1: The 20 most frequent stemmed terms.

```
set.seed(3020)
wordcloud(names(w), w, min.freq = 10, max.words = 100,
          random.order = FALSE, colors = brewer.pal(8, "Dark2"))
```

The most frequent term is “**use**” (222 times), followed by *can*, *new*, *model*, *will* and *data*. Terms such as *tool*, *work*, *research* and *engin* show that much of the discussion is about **using AI in practice**, while *openai* reflects company news. Words such as *peopl* and *human* point to discussion of AI’s effect on people.

3.4 Post length

```
posts$post_length = nchar(posts$text)
summary(posts$post_length)
```



```
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>      8.0   138.0   211.0   201.4   279.5   302.0
```

```
hist(posts$post_length, col = "lightblue", breaks = 20,
     main = "Distribution of Post Lengths",
     xlab = "Post length (characters)", ylab = "Number of posts")
```

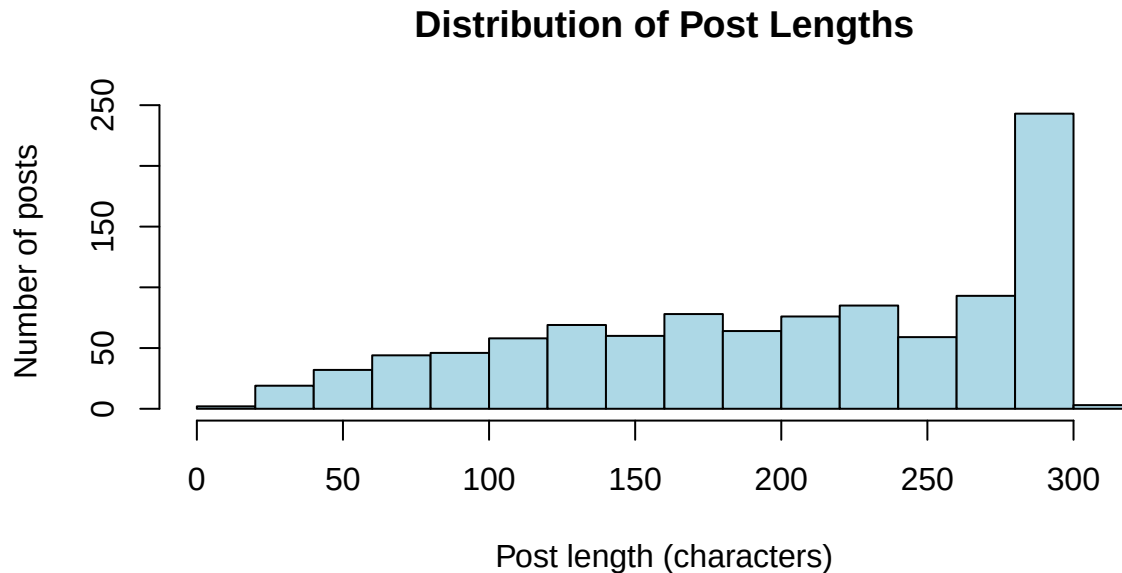


Figure 3: Distribution of post length.

The median post has 211 characters. Many posts are close to Bluesky's 300-character limit, which is typical of link and headline sharing. Short posts are often a link with a short comment.

3.5 TF-IDF representation

For clustering, each post is represented as a TF-IDF vector (Module 4 Part 2). TF-IDF gives more weight to terms that distinguish a post and less weight to terms that appear everywhere.

```
N = nrow(Bluesky.matrix)
IDF = log(N / colSums(Bluesky.matrix > 0))
TF = log(Bluesky.matrix + 1)
posts.matrix = TF %*% diag(IDF)
colnames(posts.matrix) = colnames(Bluesky.matrix)
```

4 RQ1: Topics in AI Posts (Clustering)

4.1 Representation, distance and number of clusters

Following Module 6:

- **Representation:** TF-IDF vectors, normalised to length 1 so long posts are not treated as different only because they contain more words.

- **Distance:** cosine distance, which compares *which* words are used rather than how many. It is suitable for sparse text vectors.
- **Clustering:** k-means needs Euclidean distance, so multidimensional scaling (MDS) first places the posts in 100 dimensions that preserve the cosine distances.

```
norm.posts.matrix = diag(1 / sqrt(rowSums(posts.matrix^2))) %*% posts.matrix
D = dist(norm.posts.matrix, method = "euclidean")^2 / 2 # cosine distance
mds.posts.matrix = cmdscale(D, k = 100)
```

```
n = 15
SSW = rep(0, n)
for (a in 1:n) {
  set.seed(123)
  K = kmeans(mds.posts.matrix, a, nstart = 20)
  SSW[a] = K$tot.withinss
}
plot(1:n, SSW, type = "b", pch = 19, main = "Elbow Method",
     xlab = "Number of clusters (k)", ylab = "Within-cluster sum of squares")
abline(v = 7, col = "red", lty = 2)
```

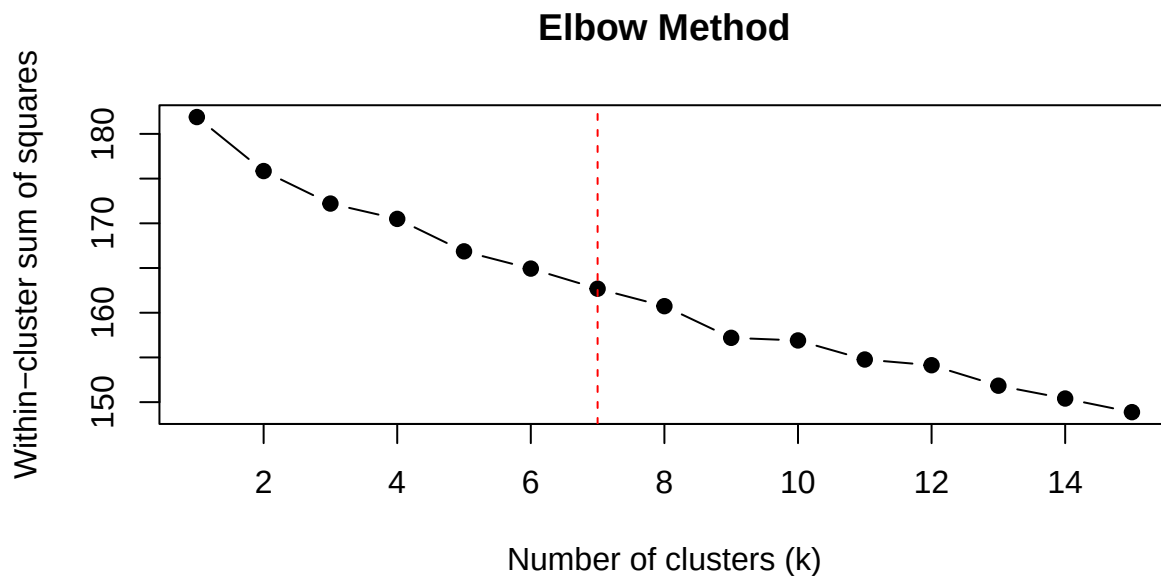


Figure 4: Elbow method: within-cluster sum of squares for $k = 1$ to 15.

The elbow curve decreases gradually and has **no sharp elbow**. Module 6 notes that this is common for short, sparse text. We therefore chose $k = 7$ because it was the smallest number of clusters that separated clearly different themes (job adverts, research, polls, Gemini, market-report spam). With fewer clusters these small themes merged into the large general groups.

```
number.of.clusters = 7
set.seed(123)
K = kmeans(mds.posts.matrix, number.of.clusters, nstart = 20)
# k-means can give slightly different solutions on different computers
# (numerical differences in MDS). To keep the report identical to the
# analysis, we use the cluster labels saved by Script 03.
```

```

saved = read.csv("data/processed/bluesky_clustered_posts.csv")
stopifnot(all(saved$uri == posts$uri))
posts$cluster = saved$cluster
table(posts$cluster)

#>
#>   1    2    3    4    5    6    7
#> 18 113    7   14 261 589   29

# Proportion of total variation explained by the saved clusters.
centres = apply(mds.posts.matrix, 2, function(v) tapply(v, posts$cluster, mean))
within.ss = sum((mds.posts.matrix - centres[posts$cluster, ])^2)
total.ss = sum(scale(mds.posts.matrix, scale = FALSE)^2)
explained = 1 - within.ss / total.ss

```

4.2 Cluster themes

For each cluster we averaged the TF-IDF weight of every term over the cluster's posts. The highest averages describe the cluster (Module 6). We also read example posts to name each theme.

```

cluster.names = c("Remote ML job adverts (bots)",
                  "AI research, books and academia",
                  "AP-NORC poll: AI developing too fast",
                  "Google Gemini, Claude and enterprise agents",
                  "Personal opinions on using AI",
                  "General AI news and link sharing",
                  "Market-report spam")

# One fixed colour per cluster, used in every plot so colours always match.
cluster.colours = c("#9AA0A6", "#7B5EA7", "#B8BCC2", "#E0A030",
                    "#1B9E77", "#2C6FB7", "#5F6368")

top.terms = rep("", number.of.clusters)
for (a in 1:number.of.clusters) {
  clusterTermWeight = colMeans(posts.matrix[posts$cluster == a, , drop = FALSE])
  top.terms[a] = paste(names(sort(clusterTermWeight, decreasing = TRUE))[1:6],
                       collapse = ", ")
}

cluster.table = data.frame(Cluster = 1:number.of.clusters,
                           Posts = as.vector(table(posts$cluster)),
                           Theme = cluster.names,
                           Top_terms = top.terms)
knitr::kable(cluster.table, caption = "Cluster sizes, themes and top TF-IDF terms.")

```

Table 1: Cluster sizes, themes and top TF-IDF terms.

Cluster	Posts	Theme	Top_terms
1	18	Remote ML job adverts (bots)	remotejob, remotework, remot, locat, alert, categori
2	113	AI research, books and academia	model, research, book, sinkan, springer, natur
3	7	AP-NORC poll: AI developing too fast	apnorc, fast, american, poll, find, develop

Cluster	Posts	Theme	Top_terms
4	14	Google Gemini, Claude and enterprise agents	gemini, claud, googl, geminai, introduc, work
5	261	Personal opinions on using AI	can, use, just, will, peopl, like
6	589	General AI news and link sharing	use, agent, data, new, que, read
7	29	Market-report spam	gpt, intellig, size, growth, openai, interact

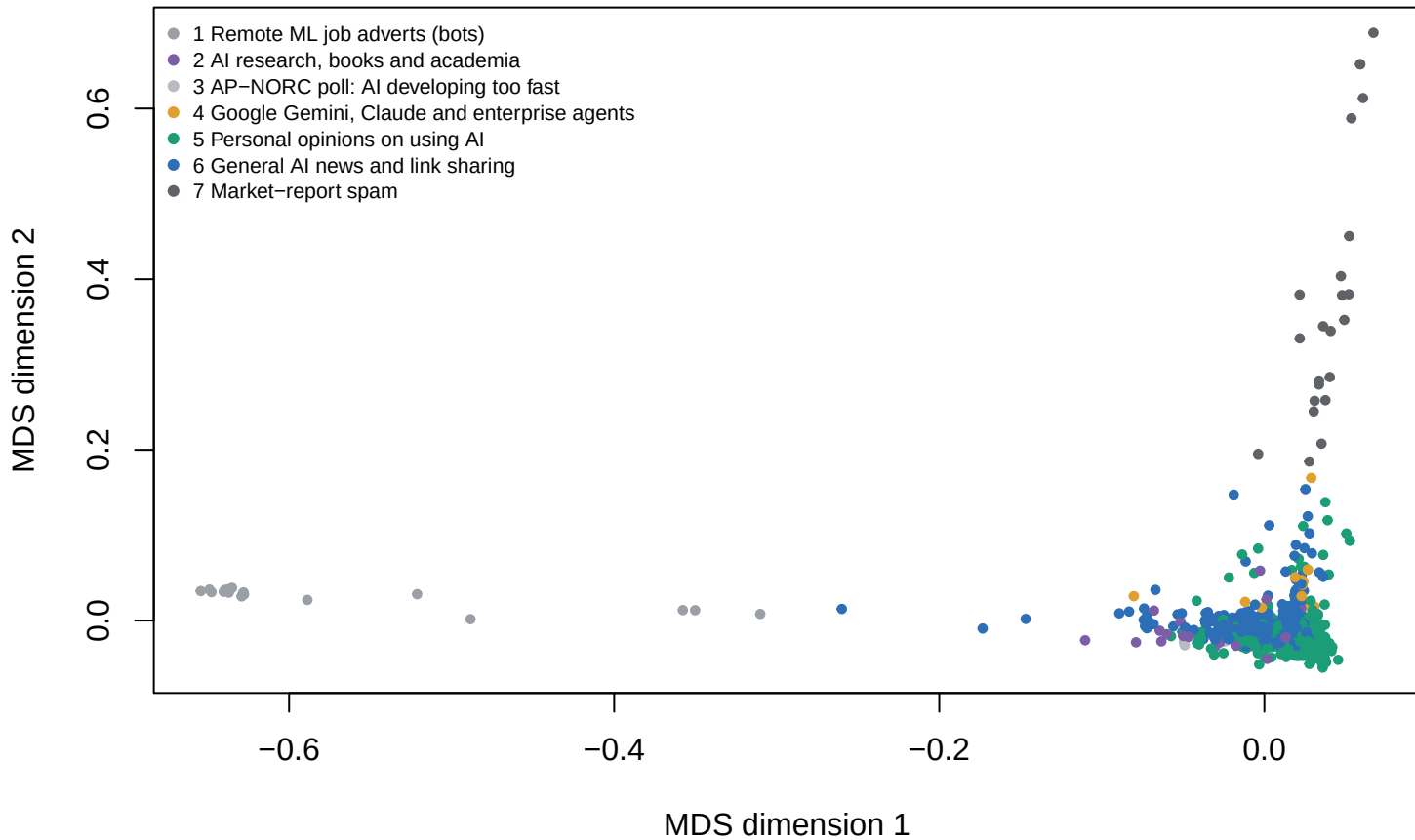
```
# One example post per cluster (shortened, non-English characters removed).
for (a in 1:number.of.clusters) {
  example = posts$text[posts$cluster == a][1]
  example = iconv(gsub("\\s+", " ", example), to = "ASCII", sub = "")
  cat("Cluster", a, ":", substr(example, 1, 85), "\n")
}
```

```
#> Cluster 1 : New Remote Job Alert! Position: Resident Engineer Company: Extreme Networks Locat
#> Cluster 2 : The evening featured Perimeter researchers Sabrina Pasterski and Tim Hsieh, who are e
#> Cluster 3 : Most Americans think artificial intelligence is developing too fast, a new AP-NORC po
#> Cluster 4 : Google Cloud introduces Gemini agent to change enterprise work ->SiliconANGLE | More
#> Cluster 5 : Worthy of a watch. An argument to slow down or pause the development of AGI, which ca
#> Cluster 6 : www.socialmediatoday.com/news/ai-favo... AI favors brands with an active social media
#> Cluster 7 : Robotics System Integration Market Size, Growth Report 2035 www.marketresearchfuture.
```

```
mds2.posts.matrix = cmdscale(D, k = 2)

plot(mds2.posts.matrix, col = cluster.colours[posts$cluster], pch = 19, cex = 0.6,
     main = "AI Discussion Clusters (2D MDS)",
     xlab = "MDS dimension 1", ylab = "MDS dimension 2")
# Legend in the empty top-left corner so it does not cover the posts.
legend("topleft", legend = paste(1:7, cluster.names), col = cluster.colours,
     pch = 19, cex = 0.7, bty = "n")
```

AI Discussion Clusters (2D MDS)



4.3 Interpretation

- Two large clusters dominate. **Cluster 6** (589 posts) is general AI news and link sharing about agents, data, OpenAI and policy. **Cluster 5** (261 posts) contains personal opinions about using AI (*can, use, just, people, think, tool*).
- **Cluster 2** groups research and academic content, including automated new-book listings.
- Three small clusters are **automated or repeated content**: remote-job bots (1), one AP-NORC poll story posted many times (3) and market-report spam (7). Together with the 87 duplicate texts removed during cleaning, this shows that a noticeable part of “AI discussion” on Bluesky is not personal conversation.
- The 2D MDS plot shows the small clusters as separate groups, while the two large clusters overlap in the centre. The large clusters share common vocabulary, so their boundary is less sharp. The k-means clusters explain only 10.6% of the total variation, so the themes are useful descriptions rather than strict categories.

4.4 Do the clusters simply repeat the search terms?

Every post was found by one of four search terms, so a concern is that the clusters only reproduce those terms. A chi-squared test of independence (Module 9) compares the search term of each post with its cluster. Some cells are small, so the p-value is simulated.

- **H0**: cluster membership is independent of the search term

- **HA:** cluster membership depends on the search term

```
keyword.table = table(posts$search_keyword, posts$cluster)
keyword.table
```

```
#>
#>
#>      artificial intelligence      1      2      3      4      5      6      7
#>      ChatGPT                  0      6      0     10     80    135    23
#>      generative AI            0     11      0      1     95    141      0
#>      machine learning         18     68      0      2     40    145      0
```

```
set.seed(123)
chisq.test(keyword.table, simulate.p.value = TRUE, B = 2000)
```

```
#>
#> Pearson's Chi-squared test with simulated p-value (based on 2000
#> replicates)
#>
#> data:  keyword.table
#> X-squared = 255.89, df = NA, p-value = 0.0004998
```

The test rejects H_0 ($p < 0.001$): topics are related to the search term (for example, the research cluster comes mostly from *machine learning* and *artificial intelligence*). However, the two largest clusters contain posts from all four search terms, so the clusters capture themes that cut across the queries rather than simply repeating them.

5 RQ2: Does Engagement Differ Between Topics?

5.1 Hypotheses and choice of test

Engagement of a post = likes + reposts + replies. The two largest clusters represent two different kinds of AI posts: **news/link sharing (Cluster 6)** and **personal opinion (Cluster 5)**. These two clusters were chosen **before** looking at engagement, because they are the two largest clusters and together contain most posts. We test whether their mean engagement differs:

- **H0:** mean engagement of news posts = mean engagement of opinion posts
- **HA:** mean engagement of news posts $\neq$ mean engagement of opinion posts (two-sided, $\alpha = 0.05$)

We use the **randomisation (permutation) test for a difference in means** from Module 10. Engagement is very skewed (most posts have 0), and the randomisation test does not assume a normal distribution. A Welch t-test is reported for comparison.

```
posts$engagement = posts$like_count + posts$repost_count + posts$reply_count
summary(posts$engagement)
```

```
#>      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>  0.000   0.000   0.000   2.646   2.000  265.000
```

```
groupA = posts$engagement[posts$cluster == 6] # news
groupB = posts$engagement[posts$cluster == 5] # opinion
c(n_news = length(groupA), n_opinion = length(groupB))
```

```
#>    n_news n_opinion
#>      589      261
```

```
c(mean_news = mean(groupA), mean_opinion = mean(groupB))
```

```
#>    mean_news mean_opinion
#>      2.432937      4.298851
```

```
meanDiff = mean(groupB) - mean(groupA)
meanDiff
```

```
#> [1] 1.865913
```

```
shuffleMean = function(groupA, groupB) {
  combined = c(groupA, groupB)
  chosen = sample(length(combined), size = length(groupA), replace = FALSE)
  mean(combined[-chosen]) - mean(combined[chosen])
}
set.seed(123)
randDist = replicate(1000, shuffleMean(groupA, groupB))
hist(randDist, col = "lightblue", main = "Randomisation Distribution",
     xlab = "Difference in mean engagement (opinion - news)",
     abline(v = meanDiff, col = "red", lwd = 2))
```

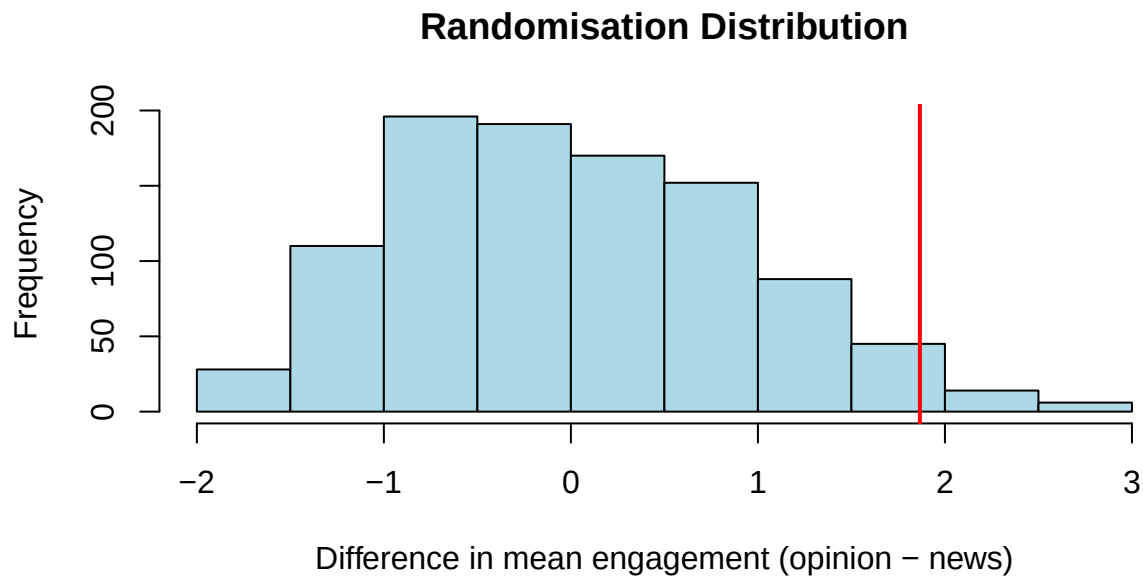


Figure 5: Randomisation distribution of the difference in means under H_0 ; the red line is the observed difference.

```
pVal = mean(abs(randDist) > abs(meanDiff))
pVal
```

```
#> [1] 0.032
```

```
tt = t.test(groupA, groupB)
tt$p.value
```

```
#> [1] 0.04591426
```

5.2 Comparing all seven topics

To check RQ2 across **all** topics, a one-way ANOVA (the `aov()` function used in Module 10) compares the mean engagement of the seven clusters. Here H_0 is that all clusters have the same mean engagement.

```
# Mean engagement of each cluster.
cluster.means = rep(0, number.of.clusters)
for (a in 1:number.of.clusters) {
  cluster.means[a] = mean(posts$engagement[posts$cluster == a])
}
round(cluster.means, 2)
```

```
#> [1] 0.00 1.42 0.43 0.29 4.30 2.43 0.17
```

```
barplot(cluster.means, names.arg = 1:7,
        col = c("grey", "grey", "grey", "grey", "lightgreen", "lightblue", "grey"),
        main = "Mean Engagement by Topic Cluster",
        xlab = "Cluster", ylab = "Mean likes + reposts + replies")
```

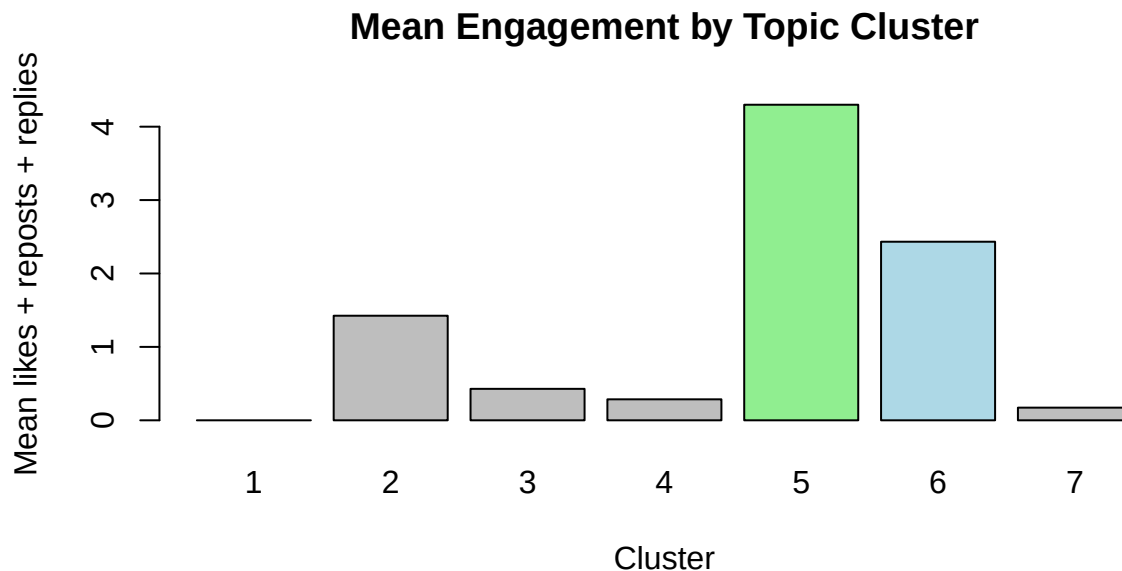


Figure 6: Mean engagement of each topic cluster; the two tested clusters are highlighted.

```
# factor() tells R that the cluster numbers are group labels.
model = aov(engagement ~ factor(cluster), data = posts)
summary(model)
```

```
#>               Df Sum Sq Mean Sq F value Pr(>F)
#> factor(cluster)    6   1324    220.7   1.694  0.119
#> Residuals       1024 133406    130.3
```

```
anova.p = summary(model)[[1]][["Pr(>F)"]][1]
```

5.3 Interpretation

- Opinion posts received on average **4.3** engagements, compared with **2.43** for news posts. The difference is 1.87.
- The randomisation p-value is **0.032** and the t-test p-value is **0.046**. Both are below 0.05, so **we reject H0**: personal opinion posts about AI attract more engagement than news and link-sharing posts. The t-test p-value is close to 0.05 and the median engagement of both groups is 0, so the difference in means is driven by a smaller number of posts with high engagement. The evidence is therefore moderate.
- The ANOVA across all seven clusters is **not significant** ($p = 0.119$). The engagement distribution is very skewed and several clusters are small, which may limit the power of this test. However, the automated clusters (1, 3, 7) have mean engagement close to zero, which supports the RQ1 finding that this content is largely ignored by users.

6 RQ3: Follow Network of AI Post Authors

6.1 Network definition

- **Node**: an author of at least one AI post in the cleaned data.
- **Edge A → B**: author A follows author B, taken from the `get_follows()` data.
- The graph is **directed** because following is one-way: A can follow B without B following A.

Most collected follows point to accounts that did not post about AI in our sample, so only follows **between** AI-post authors are kept. This makes the network directly about the AI discussion community.

```
library(igraph)
edges = read.csv("data/raw/bluesky_follow_edges.csv")
ai_users = unique(posts$author_handle)

el = as.matrix(edges[, c("from", "to")])
el = unique(el)
el = el[el[, "from"] != el[, "to"], ]
el = el[el[, "from"] %in% ai_users & el[, "to"] %in% ai_users, ]

g = graph_from_edgelist(el, directed = TRUE)
comp = components(g, mode = "weak")
```

6.2 Network properties

```
network.summary = data.frame(
  Measure = c("AI-post authors", "Authors with at least one tie (nodes)",
    "Authors with no tie (not in graph)", "Edges (follows)",
    "Density", "Weakly connected components",
    "Size of largest component"),
  Value = c(length(ai_users), vcount(g), length(ai_users) - vcount(g),
    ecount(g), round(edge_density(g), 4), comp$no, max(comp$size)))
knitr::kable(network.summary, caption = "Properties of the AI-author follow network.")
```

Table 2: Properties of the AI-author follow network.

Measure	Value
AI-post authors	803.0000
Authors with at least one tie (nodes)	340.0000
Authors with no tie (not in graph)	463.0000
Edges (follows)	480.0000
Density	0.0042
Weakly connected components	18.0000
Size of largest component	294.0000

```
plot(degree_distribution(g), type = "h", xlab = "Degree", ylab = "Proportion",
     main = "Degree Distribution of the AI Author Network")
```

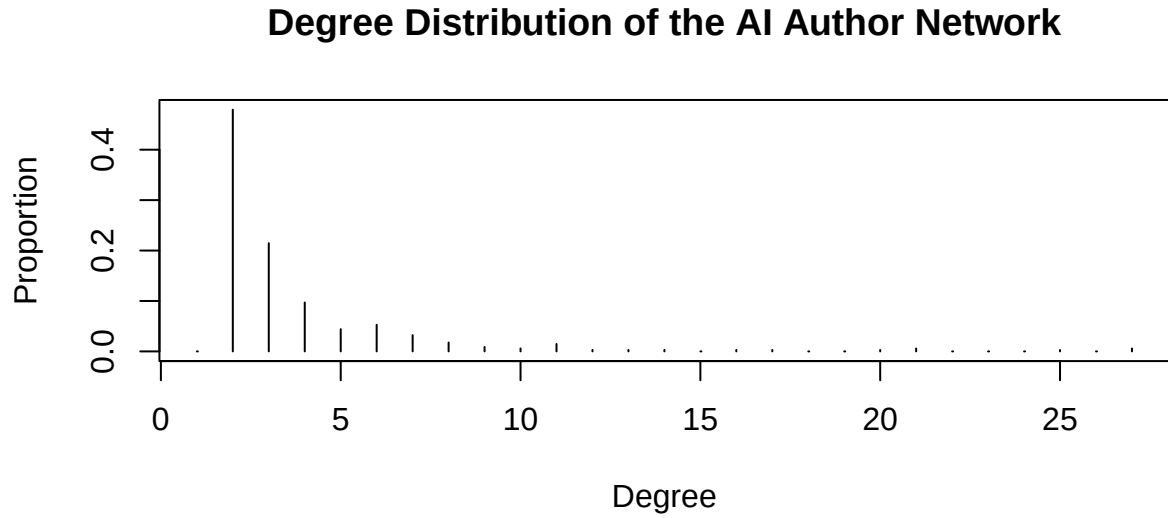


Figure 7: Degree distribution of the network.

Among the connected authors, the network is **sparse** (density 0.0042). Only 340 of 803 authors (42%) follow or are followed by another author in the sample. However, those who are connected mostly form **one large component** of 294 authors. The degree distribution is right-skewed like the Barabasi-Albert graphs in Module 7: most authors have one or two connections and a few accounts are hubs.

6.3 Visualisation

Nodes are coloured by the topic cluster of the author's first post in the sample (RQ1); authors who posted in several topics are shown with the first one. Node size shows in-degree. The five most-followed accounts are labelled.

```
author.cluster = rep(0, vcount(g))
for (a in 1:vcount(g)) {
  author.cluster[a] = posts$cluster[posts$author_handle == V(g)$name[a]][1]
}
table(author.cluster)
```

```
#> author.cluster
```

```
#> 1 2 3 4 5 6 7
#> 2 28 4 3 90 207 6
```

```
in_degree = degree(g, mode = "in")
top5 = names(sort(in_degree, decreasing = TRUE))[1:5]
```

```
labels = ifelse(V(g)$name %in% top5, V(g)$name, NA)
par(mar = c(0, 0, 2, 0))
set.seed(3020)
plot(g, layout = layout_with_fr(g),
     vertex.size = 1.5 + sqrt(in_degree) * 1.5,
     vertex.color = cluster.colours[author.cluster], vertex.frame.color = "white",
     vertex.label = labels, vertex.label.cex = 0.7, vertex.label.color = "black",
     vertex.label.dist = 1.5, vertex.label.degree = -pi / 2,
     edge.arrow.size = 0.1, edge.color = "grey70",
     main = "Follow Network of AI Post Authors")
legend("bottomleft", legend = paste(1:7, cluster.names), col = cluster.colours,
     pch = 19, cex = 0.6, bty = "n")
```

Subgraph. As in Module 7, the subgraph keeps authors with **degree greater than 1**. This removes 163 loosely attached authors and shows the core of the community more clearly. Centrality is still calculated on the full graph.

```
keep = degree(g) > 1
g2 = induced_subgraph(g, V(g)[keep])
c(nodes = vcount(g2), edges = ecoun(g2))
```

```
#> nodes edges
#> 177 325
```

```
labels2 = ifelse(V(g2)$name %in% top5, V(g2)$name, NA)
par(mar = c(0, 0, 2, 0))
set.seed(3020)
plot(g2, layout = layout_with_kk(g2),
     vertex.size = 2 + sqrt(degree(g2, mode = "in")) * 1.5,
     vertex.color = cluster.colours[author.cluster[keep]], vertex.frame.color = "white",
     vertex.label = labels2, vertex.label.cex = 0.7, vertex.label.color = "black",
     vertex.label.dist = 1.5, vertex.label.degree = -pi / 2,
     edge.arrow.size = 0.15, edge.color = "grey70",
     main = "Authors with Degree > 1")
legend("bottomleft", legend = paste(1:7, cluster.names), col = cluster.colours,
     pch = 19, cex = 0.6, bty = "n")
```

6.4 Centrality

Three centrality measures were calculated on the full graph:

- **In-degree:** the number of AI authors who follow the account (popularity).
- **PageRank (Module 8):** an account is important if it is followed by other important accounts. As in the lab, the random surfer follows a link 80% of the time (**damping** = 0.8).
- **Betweenness:** how often an account lies on the shortest paths between other accounts (a possible bridge).

Follow Network of AI Post Authors

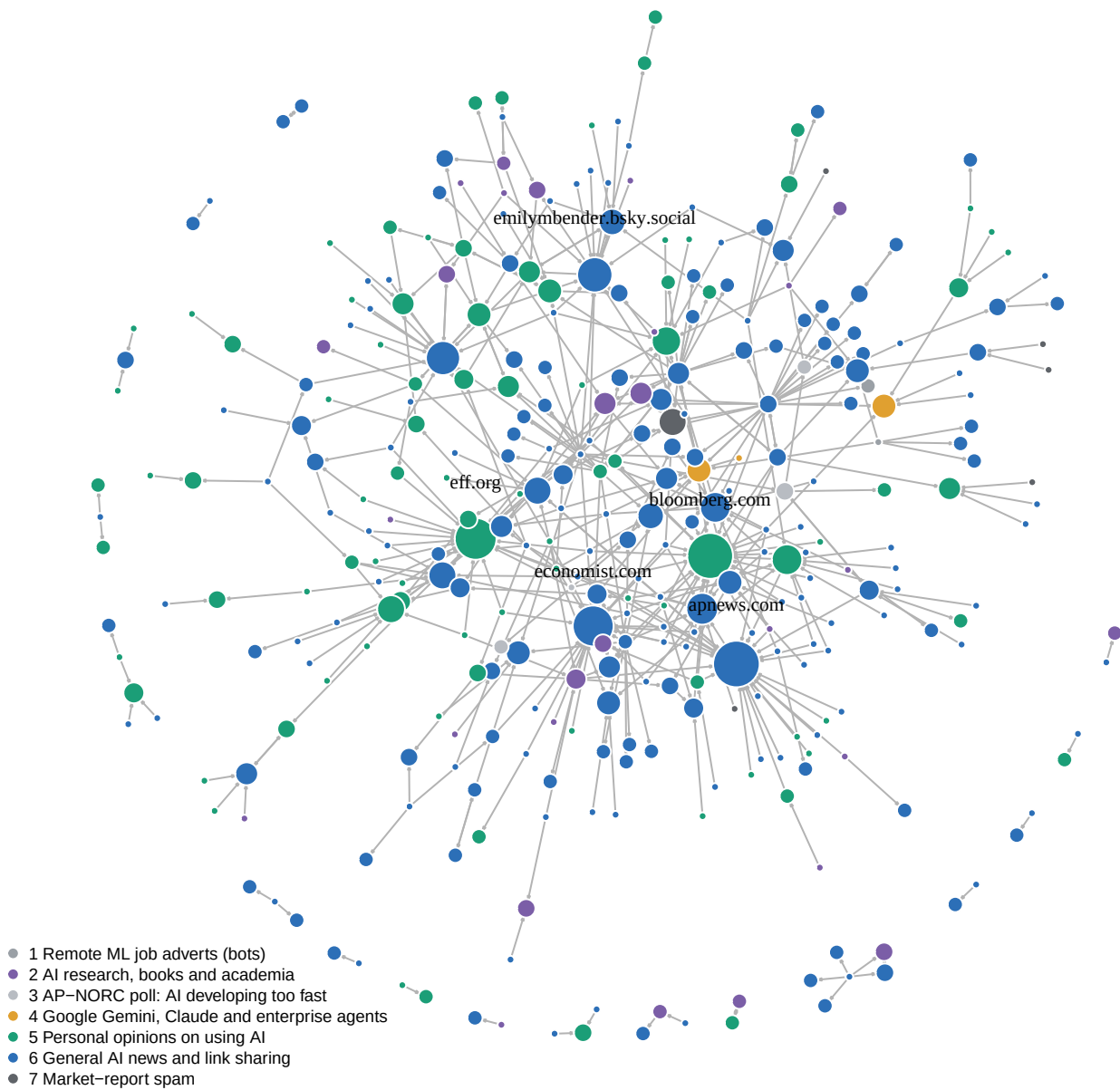


Figure 8: Full follow network of AI-post authors (authors with at least one tie).

Authors with Degree > 1

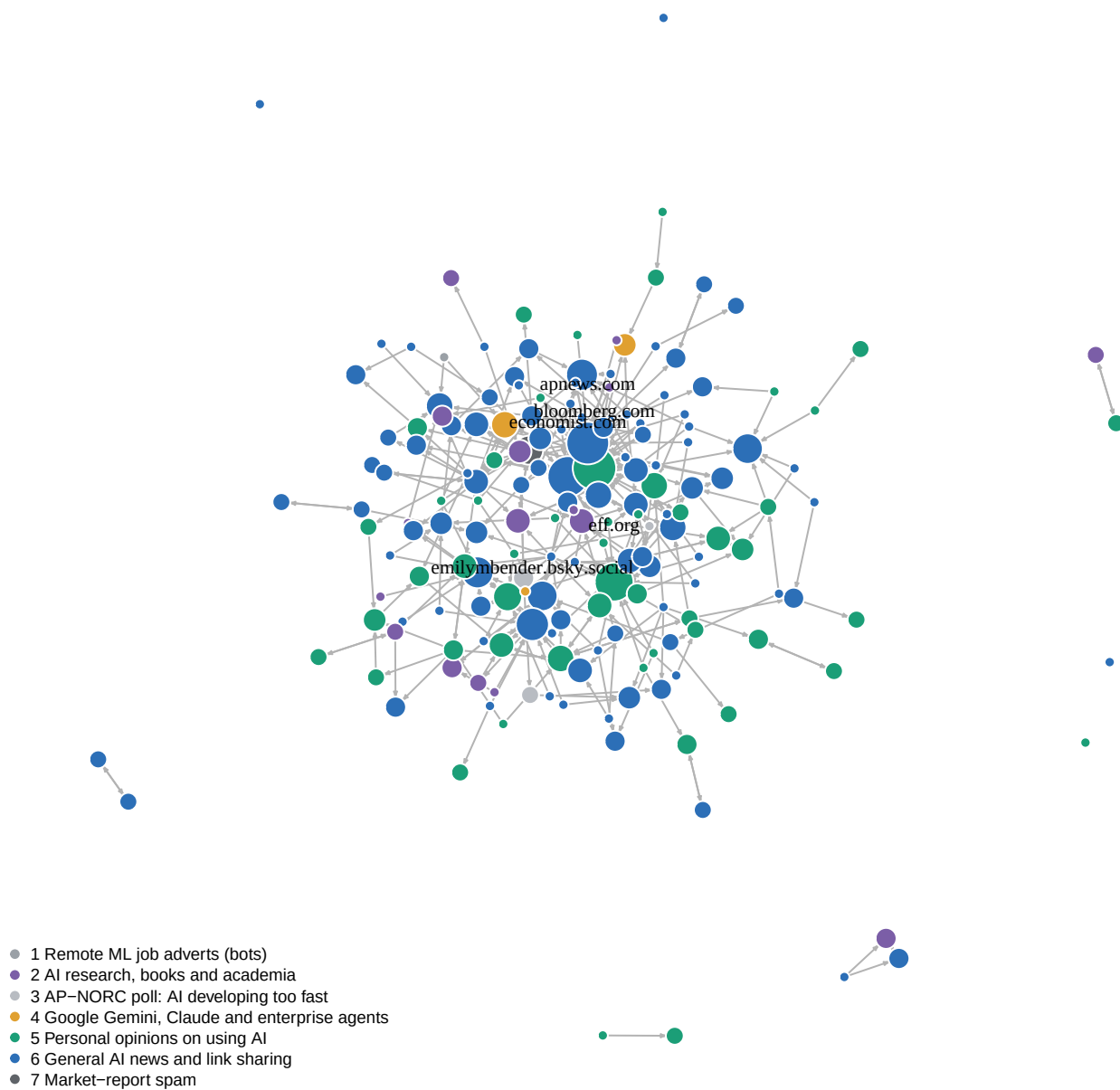


Figure 9: Subgraph of authors with degree greater than 1.

```

pr = page_rank(g, damping = 0.8)$vector
btw = betweenness(g, directed = TRUE)

top_in = head(sort(in_degree, decreasing = TRUE), 5)
top_pr = head(sort(pr, decreasing = TRUE), 5)
top_btw = head(sort(btw, decreasing = TRUE), 5)

knitr::kable(data.frame(
  Rank = 1:5,
  In_degree = paste0(names(top_in), " (", top_in, ")"),
  PageRank = paste0(names(top_pr), " (", round(top_pr, 3), ")"),
  Betweenness = paste0(names(top_btw), " (", round(top_btw), ")")),
  caption = "Top five accounts for each centrality measure.")

```

Table 3: Top five accounts for each centrality measure.

Rank	In_degree	PageRank	Betweenness
1	apnews.com (26)	ketanjoshi.co (0.06)	vinkius-mcp-ai.bsky.social (824)
2	bloomberg.com (25)	nposegay.northsky.social (0.05)	colin-fraser.net (620)
3	eff.org (20)	savasavasava.myatproto.social (0.033)	bigearthdata.ai (556)
4	economist.com (19)	mattseybold.bsky.social (0.021)	bloomberg.com (482)
5	emilymbender.bsky.social (13)	apnews.com (0.019)	pauljdavies.bsky.social (448)

6.5 Interpretation

- **In-degree** is led by news outlets (**apnews.com**, **bloomberg.com**, *economist.com*), the digital-rights organisation *eff.org* and the AI-critical linguist *emilymbender*. Within this sample, authors most often follow established organisations.
- **PageRank** ranks a different set of accounts, mainly individual commentators (for example **ketanjoshi.co**). These accounts are followed by other well-connected authors, even if they are not followed by many authors overall.
- **Betweenness** identifies bridges such as **vinkius-mcp-ai.bsky.social** and **colin-fraser.net**, which connect otherwise separate parts of the network. *bloomberg.com* is high in both in-degree and betweenness.
- Most connected authors posted in the general news (207) or opinion (90) clusters. Very few authors from the automated clusters (job bots, market-report spam) are connected to other AI authors.
- The three measures rank different accounts, so “central” depends on the definition: popular (in-degree), endorsed by important accounts (PageRank) or bridging (betweenness).

6.6 Linking the network to engagement (RQ2 and RQ3)

RQ2 showed that engagement differs between topics. A natural follow-up is whether it also differs with an author’s position in the follow network. We apply the same randomisation test (Module 10) to compare posts by **connected authors** (at least one follow tie with another AI author) and **isolated authors**.

- **H0**: mean engagement is equal for connected and isolated authors
- **HA**: mean engagement differs (two-sided, $\alpha = 0.05$)

```

connected = posts$author_handle %in% V(g)$name
isolatedEng = posts$engagement[!connected]
connectedEng = posts$engagement[connected]
c(n_isolated = length(isolatedEng), n_connected = length(connectedEng))

```

```
#> n_isolated n_connected
#>          581          450
```

```
c(mean_isolated = mean(isolatedEng), mean_connected = mean(connectedEng))
```

```
#> mean_isolated mean_connected
#>          1.500861          4.124444
```

```
netDiff = mean(connectedEng) - mean(isolatedEng)
set.seed(123)
netDist = replicate(1000, shuffleMean(isolatedEng, connectedEng))
hist(netDist, col = "lightblue", xlim = range(c(netDist, netDiff)),
     main = "Randomisation Distribution: Connected vs Isolated Authors",
     xlab = "Difference in mean engagement (connected - isolated)")
abline(v = netDiff, col = "red", lwd = 2)
```

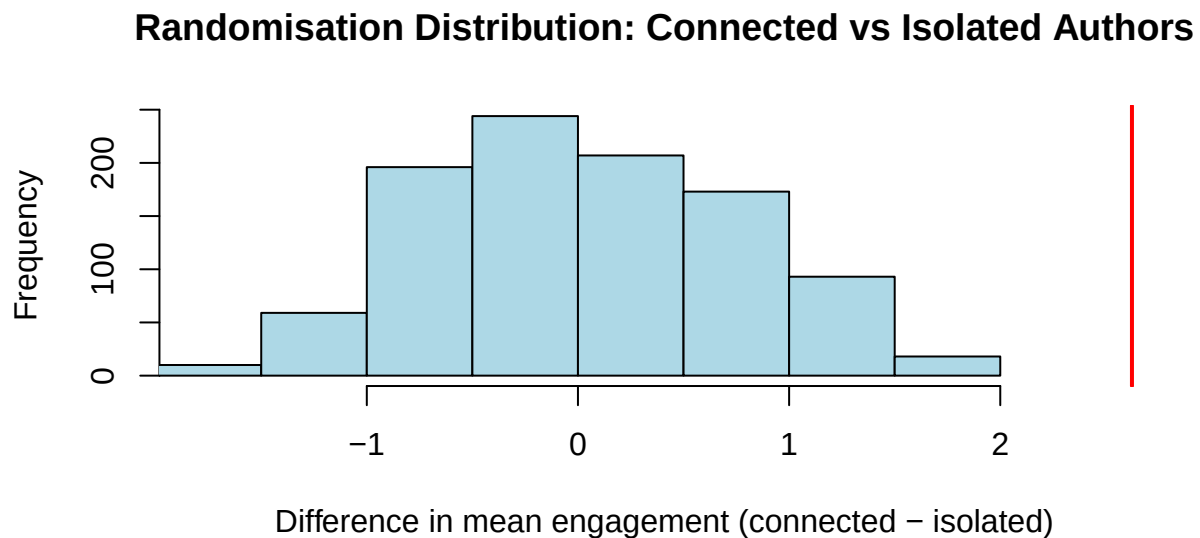


Figure 10: Randomisation distribution for connected vs isolated authors; the red line is the observed difference.

```
netP = mean(abs(netDist) > abs(netDiff))
netP
```

```
#> [1] 0
```

```
# Among connected authors: followed by at least one other AI author?
followed = names(in_degree)[in_degree > 0]
followedEng = posts$engagement[posts$author_handle %in% followed]
notFollowedEng = posts$engagement[connected & !(posts$author_handle %in% followed)]
c(mean_followed = mean(followedEng), mean_not_followed = mean(notFollowedEng))
```

```
#> mean_followed mean_not_followed
#>          6.106464          1.336898
```

Posts by connected authors received on average **4.12** engagements, compared with **1.5** for isolated authors. None of the 1,000 shuffled differences was as large as the observed difference ($p < 0.001$), so **we reject H0**. Among connected authors, those followed by at least one other AI author averaged 6.11 engagements, compared with 1.34 for those who follow others but are not followed. This links the two analyses: **engagement is higher for authors who are embedded in the AI community**. This is an association within the sample, not evidence that follows cause engagement; for example, active and established accounts may both attract followers and receive more likes.

6.7 Effect of data collection on the network

- About 300 follows were collected per author. For authors who follow many accounts, some edges to other AI authors are missing, so missing follows may underestimate in-degree and distort betweenness.
- The sample covers about one day of posts, so many authors in the AI community are not in the sample at all.
- Keeping only follows between AI-post authors makes the network relevant to RQ3 but removes 463 authors with no ties from the graph.
- A follow shows interest, not an actual conversation about AI.

7 Overall Findings

1. **What is discussed (RQ1):** AI discussion on Bluesky in this sample is dominated by **general AI news and link sharing** (57% of posts) and **personal opinions about using AI** (25%). A noticeable share is **automated or repeated content** (job bots, market-report spam, the same headline posted many times), which forms its own clusters.
2. **What gets engagement (RQ2):** opinion posts receive significantly more engagement than news posts ($p = 0.032$). Automated content receives almost no engagement. Across all seven topics, the one-way ANOVA did not find a significant difference.
3. **Who is connected (RQ3):** the authors form a **sparse** network. Most authors are not connected to each other, but the connected authors mainly form one large component. News outlets and organisations are the most followed accounts, while individual commentators rank highest on PageRank and betweenness. Automated accounts are almost absent from the connected core.
4. **How engagement and the network connect (RQ2 and RQ3):** posts by authors who are connected to other AI authors receive significantly more engagement than posts by isolated authors ($p < 0.001$), and authors followed by other AI authors receive the most.

Together, the results suggest that, in this sample, **human opinion posts about AI receive more engagement than shared news**. In the follow network, authors most often follow established organisations, while individual commentators occupy structurally central positions; because the network is built from follows, it shows who pays attention to whom, not where conversations take place. Engagement is concentrated among authors who are embedded in the community, while automated AI-related content makes up part of the volume but receives very little engagement and is rarely connected to other authors.

8 Limitations

- **Sampling:** keyword search over about 25 hours, sorted by latest. The results describe this sample, not all of Bluesky or all AI discussion.
- **Text processing:** non-English posts lose most of their words after conversion to ASCII, and stemming produces shortened terms (e.g. *peopl*).
- **Clustering:** there was no clear elbow, k-means depends on the chosen k and the random start (fixed with `set.seed`), and the two large clusters overlap.
- **Hypothesis test:** engagement is very skewed with many zeros, and the t-test p-value is close to 0.05.
- **Network:** limited follows per author, a sparse graph with 18 components, and follows do not show actual interaction.

9 References

- Gruber, J. B., Guinaudeau, B., & Votta, F. (2024). *atrrrr: Wrapper for the AT Protocol behind Bluesky* [R package]. <https://CRAN.R-project.org/package=atrrrr>
- Feinerer, I., & Hornik, K. (2024). *tm: Text mining package* [R package]. <https://CRAN.R-project.org/package=tm>
- Csardi, G., & Nepusz, T. (2006). The igraph software package for complex network research. *InterJournal, Complex Systems*, 1695.
- Fellows, I. (2018). *wordcloud: Word clouds* [R package]. <https://CRAN.R-project.org/package=wordcloud>
- COMP3020 Social Web Analytics lab materials, Modules 4–10, Western Sydney University (2026).